Project Background and Problem Context

Realtime ASR Project Background CS5296 Group 7

Goal: Build a practical, realtime speech-to-text web system that supports downstream understanding.

Why this project matters

- Traditional transcription workflows are often offline and slow.
- Real deployments need low latency, stability, and easy operations.
- Teams also need immediate value after transcription, such as summary and rewriting.

Project objectives

- Stream browser microphone audio to backend in realtime.
- Produce live transcripts using Vosk ASR.
- Provide one-click post-processing: summarize and rewrite.
- Support cloud deployment with observability.

System Background and Architecture Rationale

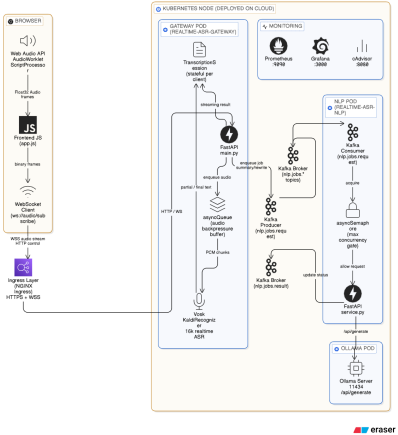

Figure 1: End-to-end architecture of Live ASR stream processing pipeline

Summary of Tuning Results and Final Configuration

Deployment targets

- Docker Compose for fast local iteration.
- Kubernetes (k3s) for cloud deployment on Alibaba ECS.

Operational concerns observed in practice

- HTTPS and ingress configuration for secure microphone access.
- Resource pressure from larger ASR models under limited ECS CPU and memory.
- WebSocket session stability under real network and load conditions.

Supporting platform capabilities

- Prometheus and Grafana for monitoring and performance visibility.
- Configurable runtime knobs (queue size, concurrency, model path).
- Scripted deployment to improve reproducibility and onboarding.

Deployment Background and Engineering Constraints

Practical implication

- Simulate 2-node deployment with KIND (Kubernetes in Docker) and load testing script.
- Which configuration gave the best balance of responsiveness and stability under constrained resources?

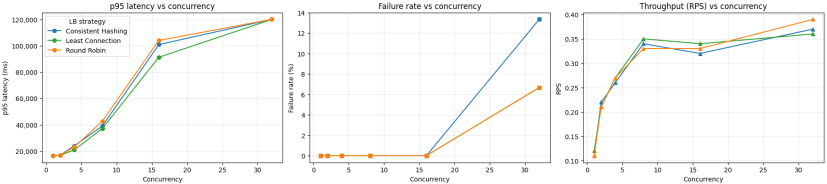

Figure 2: Benchmark comparison of single-server double-node tuning candidates

Performance Tuning Results for Single-Server Deployment (Cont.)

Additional profiling evidence is shown in Figure 3 and Figure 4.

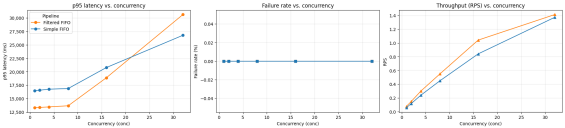

Figure 3: Chunk queue method comparison under sustained traffic

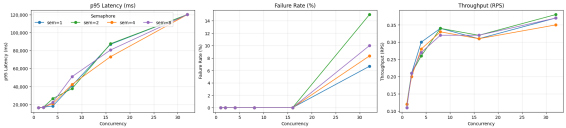

Figure 4: Semaphore sweep: throughput vs. stability trade-off

Performance Tuning Results for Single-Server Deployment

What we tested

- Load balancing algorithms across gateway replicas.
- Different audio chunk queue handling methods.
- Concurrency control with semaphore limits.

Key finding

- For stateless single-server single-node deployment, the most stable setup is:
- Semaphore = 1
- Load balancing algorithm = least connection